CipherCompute: Statistics and Inference on Encrypted Data with CKKS

A technical whitepaper. It assumes linear algebra and modular arithmetic at the level of a strong undergraduate, and no prior knowledge of homomorphic encryption. Every precision, performance and load figure in it was measured on the system described, and the tables are generated from those measurements by scripts in the repository. Security bounds are quoted from the cited standard.

1. The problem

A hospital wants a statistic — a mean blood pressure, a risk score from a trained model — computed by a service it does not fully trust. Encryption at rest and in transit does not help: to compute, the service must decrypt, and at that moment the data is exposed to whoever controls the machine.

Fully homomorphic encryption (FHE) removes that moment. The service receives ciphertexts and computes on them directly; the output is another ciphertext that only the data owner can decrypt. CipherCompute packages one FHE scheme, CKKS, as a multi-tenant service: a client SDK that keeps the secret key, an API and worker that only ever hold public keys, and the operational machinery (depth budgeting, integrity checks, audit) that makes it usable.

2. From lattices to approximate arithmetic

2.1 Learning With Errors

Pick a secret vector $\mathbf{s}$. Publish many pairs $(\mathbf{a}_i, b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \bmod q)$ with random $\mathbf{a}_i$ and small random "errors" e_i. Without the errors, Gaussian elimination recovers $\mathbf{s}$. With them, recovering $\mathbf{s}$ — or even distinguishing the pairs from random — is believed hard, also for quantum computers. This is the **Learning With Errors (LWE)** problem.

To hide a message m, add it to such a sample: $c = (\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e + m)$. Whoever knows $\mathbf{s}$ subtracts $\langle \mathbf{a}, \mathbf{s} \rangle$ and is left with $m + e$: the message, plus a little noise.

2.2 Rings make it efficient

LWE ciphertexts are large. **Ring-LWE** replaces vectors by polynomials in $R_q = \mathbb{Z}_q[X]/(X^N + 1)$, with N a power of two. One ring element carries N coefficients, and multiplication of polynomials (computed with the number-theoretic transform in $O(N \log N)$) does the work of an $N \times N$ matrix-vector product. CipherCompute uses $N \in \{8192, 16384, 32768\}$.

A ciphertext is a pair $(c_0, c_1) \in R_q^2$ with $c_0 + c_1 \cdot s \approx m$.

2.3 Adding and multiplying ciphertexts

- **Addition** is component-wise: $(c_0 + c_0', c_1 + c_1')$ decrypts to $m + m'$; the noises add.
- **Multiplication** of (c_0, c_1) by (c_0', c_1') gives three components (d_0, d_1, d_2) that decrypt with $(1, s, s^2)$ to $m \cdot m'$. **Relinearisation** uses a public *relinearisation key* — an encryption of s^2 under s — to fold d_2 back into a normal two-component ciphertext.
- **Rotations** of the encoded vector (needed for sums and dot products) use public *Galois keys*.

None of these need the secret key. That is the whole point: the server holds public, relinearisation and Galois keys, and computes.

2.4 CKKS: encoding real numbers

Integers mod q are a poor fit for statistics. **CKKS** (Cheon, Kim, Kim and Song, 2017) instead treats the noise as part of the approximation error of fixed-point arithmetic:

1. **Slots.** A vector **z** of up to N/2 complex numbers is mapped by an inverse "canonical embedding" (essentially an inverse FFT) to a polynomial with real coefficients. Operations on polynomials then act *slot-wise* on **z** — one homomorphic multiplication multiplies thousands of numbers at once (SIMD).
2. **Scale.** The coefficients are multiplied by a large **scale** Δ (we use 2^{25}–2^{40}) and rounded to integers. Decryption divides by Δ again, so the rounding and the encryption noise become small errors in the low-order bits of the result.

2.5 Rescaling and the modulus chain

Multiplying two encodings multiplies their scales: $\Delta \cdot \Delta = \Delta^2$. Left alone, the scale explodes after a few multiplications. CKKS **rescales**: it divides the ciphertext by (approximately) Δ, which also divides the modulus q by one prime factor. So q is chosen as a product of primes $q = p_0 \cdot p_1 \cdot \ldots \cdot p_L$ — the **coefficient modulus chain** — and every multiplication consumes one prime. When only p_0 remains, no further multiplication is possible.

The number of primes available for rescaling is the **multiplicative depth** of the parameter set. This is CKKS's "noise budget": not a quantity of noise, but a count of levels. A degree-3 polynomial needs 2 levels; computing a logistic regression score needs 3.

2.6 Security: why the primes are bounded

Larger q leaves more room for depth but makes the lattice problem easier. The **Homomorphic Encryption Security Standard** tabulates, for each N and target security, the maximum $\log_2 q$:

N	128-bit	192-bit	256-bit
8192	218	152	118
16384	438	305	237
32768	881	611	476

Every CipherCompute parameter set is checked against this table when it is constructed; an insecure combination cannot be built, and the server re-checks every key bundle it receives.

3. System design

3.1 The key boundary

The SDK generates the key ring locally and stores the secret key encrypted (AES-256-GCM, key derived with Argon2id). The only key object that is ever serialised for upload hard-codes "no secret key", and the server's single entry point for key material, `PublicKeyBundle.load`, rejects any context that can decrypt. A CI test scans the server source for any symbol that could touch secret material. The guarantee is therefore structural, not procedural: there is no code path by which the service could decrypt.

3.2 Depth planning before execution

Each operation declares its depth cost; the API sums the costs of a job, adds the levels its inputs have already consumed, and refuses the job *before* queueing it if the chain is too short — naming the smallest standard parameter set that would work. The declared costs are not trusted: regression tests measure, for every operation and parameter set, the levels the result actually consumed, two independent ways, and fail CI on any drift.

3.3 Integrity against a malicious server

FHE gives confidentiality, not correctness: a malicious server could return a well-formed wrong ciphertext. CipherCompute narrows this: every upload's SHA-256 is recorded and re-checked by the worker, every job carries a manifest hash committing to operation, arguments and inputs, and the SDK can recompute a random sample of jobs in plaintext (`verify_sample`). This detects systematic cheating probabilistically; it is not a proof of correct computation, and the threat model says so.

4. Precision: what "approximately equal" means

CKKS results are approximate, so "correct" needs a definition. For inputs **x**, plaintext arguments **a** and exact result **y**, let $M = \max(1, |\mathbf{x}|_\infty, |\mathbf{a}|_\infty, |\mathbf{y}|_\infty)$. We call a result correct when $|\text{result} - \mathbf{y}|_\infty \le M \cdot 2^{-b}$, where b is the **precision in bits** for the parameter set at the depth the computation consumed.

Instead of quoting a theoretical bound, CipherCompute *measures* b: a calibration script runs every operation on random, zero and constant inputs at several magnitudes and records the worst error per level; a 16× safety margin (4 bits) is subtracted. Property-based tests (Hypothesis) then check decrypt(op(enc(x))) against op(x) within exactly that tolerance, for every operation and parameter set.

A finding worth knowing. The first calibration used one key ring per parameter set. Hypothesis immediately found inputs whose error was 13× larger than anything calibration had seen. Tracing it showed the cause was not the inputs but the **key ring**: the noise floor of rotation-heavy operations (sum, mean, dot product — 12 rotations each at N = 8192) varies by 14–26× between fresh key rings with identical parameters, with outliers near 100×. A precision bound measured on one key is not a bound. Calibration now samples 12 key rings per N = 8192 set and 4 per N = 16384 set.

The same data showed that one number per level is too coarse. Slot-wise operations (add, multiply, element-wise products, regression on feature-major layouts) never key-switch, and their error is set by the scale; rotating operations (sum, mean, dot product, variance, matrix-vector) pay the key-switching noise floor at every level. Precision is therefore recorded per class, and every job reports the class that applies to the operations it ran. A computation at depth L may contain any operation measured at a shallower depth, so the bound at L is the worst measurement at or below L; a level no calibrated operation lands on exactly is also bounded by the next measured level and is marked * in the table, as is extrapolation beyond the deepest measured level.

The measured precision per level (bits, after the 4-bit margin; 0 = no usable precision) for each standard set:

N	security	profile	primes	levels	slot-wise ops: bits at level 0, 1, …	rotating ops: bits at level 0, 1, …	key rings
8192	128	balanced	[60, 40, 40, 60]	2	22 · 18 · 15*	14 · 14 · 14	12
8192	128	deep	[45, 30, 30, 30, 30, 45]	4	12 · 9 · 9* · 9 · 9	4 · 4 · 4 · 4 · 4	12
8192	192	balanced	[45, 30, 30, 45]	2	12 · 10 · 7*	4 · 4 · 4	12
8192	192	deep	[38, 25, 25, 25, 38]	3	7 · 5 · 4* · 4	0 · 0 · 0 · 0	12
8192	256	balanced	[36, 23, 23, 36]	2	5 · 1 · 0*	0 · 0 · 0	12
16384	128	balanced	[60, 40, 40, 40, 40, 40, 40, 60]	6	21 · 15 · 15* · 15 · 15 · 12* · 9*	12 · 12 · 12 · 12 · 12 · 9* · 6*	4
16384	128	deep	[50, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 50]	11	11 · 8 · 8* · 8 · 7 · 4* · 1* · 0* · 0* · 0* · 0* · 0*	3 · 3 · 3 · 3 · 3 · 0* · 0* · 0* · 0* · 0* · 0* · 0*	4
16384	192	balanced	[55, 35, 35, 35, 35, 35, 55]	5	17 · 12 · 12* · 12 · 12 · 9*	7 · 7 · 7 · 7 · 7 · 4*	4
16384	192	deep	[45, 30, 30, 30, 30, 30, 30, 30, 45]	7	11 · 9 · 9* · 9 · 8 · 5* · 2* · 0*	4 · 4 · 4 · 4 · 4 · 1* · 0* · 0*	4
16384	256	balanced	[43, 29, 29, 29, 29, 29, 43]	5	10 · 7 · 7* · 7 · 7 · 4*	3 · 2 · 2 · 2 · 2 · 0*	4
16384	256	deep	[38, 26, 26, 26, 26, 26, 26, 38]	6	7 · 5 · 5* · 5 · 4 · 1* · 0*	0 · 0 · 0 · 0 · 0 · 0* · 0*	4

These are deliberately pessimistic numbers. Each is the worst case over many key rings and input patterns

(all-zero and constant vectors included), relative to the largest magnitude involved, minus a further 4 bits. Typical errors are several bits better. They are still the numbers a client can rely on, and three consequences follow.

First, slot-wise work is cheap in precision: the balanced N = 8192, 128-bit set keeps 22 bits fresh and 18 after a multiplication. Second, anything that rotates — sums, means, dot products, variances — sits on an absolute floor of about 2^{-14} of the largest magnitude at that set. That is ample for vital signs, but not for quantities that need six significant digits. Third, higher security is paid for in precision. The 192- and 256-bit sets keep 0–4 bits after a rotation at N = 8192, and at most 7 at N = 16384. The parameter advisor accepts a minimum precision and refuses, naming the reason, when no standard set provides it. For rotation-heavy statistics where precision matters, 128-bit security at N = 8192 or N = 16384 is the practical choice. The case study in §6 lays its data out by column so that its regression stays slot-wise.

5. Performance

Measured on Intel64 Family 6 Model 154 Stepping 3, GenuineIntel (20 logical CPUs, 15.7 GB RAM), Windows 11, Python 3.12.16, TenSEAL 0.3.18; one process per parameter set, median of repeated runs.

N	security	profile	depth	slots	keygen	public bundle	ciphertext	encrypt	decrypt	fresh precision (slot-wise / rotating)
8192	128	balanced	2	4096	0.233 s	35.3 MB	331.7 KB	3.24 ms	0.94 ms	22 / 14 bits
8192	128	deep	4	4096	0.501 s	65.3 MB	405.9 KB	4.43 ms	1.47 ms	12 / 4 bits
8192	192	balanced	2	4096	0.233 s	28.1 MB	257.6 KB	3.46 ms	1.0 ms	12 / 4 bits
8192	192	deep	3	4096	0.356 s	39.4 MB	291.5 KB	3.96 ms	1.12 ms	7 / 0 bits
8192	256	balanced	2	4096	0.231 s	22.8 MB	211.1 KB	3.25 ms	0.85 ms	5 / 0 bits
16384	128	balanced	6	8192	1.915 s	325.9 MB	1449.9 KB	11.54 ms	5.08 ms	21 / 12 bits
16384	128	deep	11	8192	5.282 s	682.4 MB	1860.2 KB	19.55 ms	9.45 ms	11 / 3 bits
16384	192	balanced	5	8192	1.444 s	222.5 MB	1118.4 KB	11.34 ms	3.9 ms	17 / 7 bits
16384	192	deep	7	8192	2.427 s	320.6 MB	1253.2 KB	13.55 ms	6.65 ms	11 / 4 bits
16384	256	balanced	5	8192	1.444 s	189.1 MB	944.4 KB	9.62 ms	3.85 ms	10 / 3 bits
16384	256	deep	6	8192	1.926 s	226.0 MB	996.5 KB	13.19 ms	5.67 ms	7 / 0 bits

Operation latency at N = 8192 (milliseconds; – = refused, not enough depth):

operation	128-bit balanced	128-bit deep	192-bit balanced	192-bit deep	256-bit balanced
vector_add	0.1	0.2	0.2	0.1	0.1
scalar_multiply	0.5	0.9	0.5	0.7	0.6
elementwise_multiply	2.2	4.9	2.1	3.1	2.1
dot_product	13.3	32.8	13.4	21.8	13.7
sum	17.9	40.9	18.7	29.0	18.2
mean	18.3	42.3	18.6	31.4	18.6
variance	33.6	78.1	34.2	58.9	34.1
weighted_average	12.7	31.1	13.2	20.6	13.0
linear_regression	2.1	4.1	2.0	3.3	2.1
logistic_regression deg3	—	15.4	—	10.9	—
logistic_regression deg5	—	20.5	—	—	—
matvec_multiply (16x64)	67.0	156.5	69.0	98.7	65.9

Operation latency at N = 16384 (milliseconds; – = refused, not enough depth):

operation	128-bit balanced	128-bit deep	192-bit balanced	192-bit deep	256-bit balanced	256-bit deep
vector_add	0.9	1.3	0.5	0.8	0.5	1.1
scalar_multiply	3.4	5.5	2.8	3.5	2.4	3.5
elementwise_multiply	15.5	35.6	12.0	20.1	11.8	16.3
dot_product	145.6	376.7	100.7	178.3	99.8	146.0
sum	173.9	384.3	118.4	200.5	128.9	156.8
mean	158.5	385.3	121.8	194.2	117.5	154.1
variance	303.5	781.1	231.6	385.9	237.2	303.8
weighted_average	122.5	331.9	93.0	158.3	96.3	123.8
linear_regression	13.0	23.7	9.9	16.0	10.6	12.8
logistic_regression deg3	50.3	111.6	40.0	62.6	41.0	53.0
logistic_regression deg5	67.5	184.2	53.4	87.9	55.6	70.7
matvec_multiply (16x64)	538.1	1216.3	434.5	654.3	440.2	534.7

Reading the table: costs grow roughly with N log N per polynomial operation and linearly with the number of primes; rotations (sum, mean, dot product, variance) dominate the statistics. Public key bundles are large because the Galois keys for all rotations are included — tens of megabytes at $N = 8192$, hundreds at $N = 16384$ — which is why the worker caches them by fingerprint.

5.1 Throughput under load

At **$N = 8192$, 128-bit security (balanced, scale 2^40)**, 20 simulated analysts sharing one organisation submitted `mean`, `dot_product` and `variance` jobs over full-width ciphertexts (4,096 slots, 332 KB each) for 5 minutes, each keeping at most 4 jobs in flight. The system under test was the single-process local stack (real API and job executor with 4 executors; SQLite, fakeredis and a local S3 server) on one 20-thread laptop that also ran the load generator. The production topology was not measured (details in the full report).

metric	result
HTTP requests	12,034, **0 failures**
jobs completed	3,680 accepted, 3,680 succeeded, 0 retried
sustained job throughput	**12.3 jobs/s** (saturated)
`POST /jobs` latency (accept, depth pre-check, enqueue)	p50 80 ms · p95 240 ms · p99 540 ms
`GET /jobs/{id}` latency	p50 32 ms · p95 130 ms · p99 320 ms
job latency, accepted → succeeded (server timestamps)	p50 1.7 s · p95 3.0 s · p99 3.2 s
of which queue wait	p50 1.4 s · p95 2.7 s
of which execution (fetch, verify, compute, store)	p50 0.26 s · p95 0.39 s

The system stayed correct under saturation: no errors, no retries, every result stored with its fingerprint and depth report. Job latency is dominated by queueing, because the test pushed one tenant past what four executors can serve. The platform's per-tenant limit is also four concurrent jobs, so adding executors would not raise this tenant's throughput. It would raise the platform's total throughput across tenants.

Full report: `loadtest/REPORT.md`.

6. Case study: encrypted cardiovascular risk

A hospital scores patients with a logistic regression model on a server it does not trust with patient data (synthetic data only; `showcase/health-risk`). Seven standardised vital signs are encrypted one *column* per ciphertext (up to 4,096 patients each); the server computes $w \cdot x + b$ with seven plaintext-ciphertext products and applies a degree-3 polynomial approximation of the sigmoid, fitted on $[-8, 8]$. The job uses 3 of the 4 levels of the $N = 8192$, 128-bit *deep* parameter set.

	value
max |encrypted − plaintext polynomial model|	2.35×10^{-4}
max |encrypted − exact sigmoid|	0.098
AUC, encrypted scores	0.861
AUC, plaintext model	0.861
server compute time, 600 patients	0.83 s

The two error figures measure different things. CKKS noise moves each score by at most 2.4×10^{-4}; the homomorphic evaluation is faithful. The polynomial sigmoid differs from the true logistic function by up to 0.098 — but it is monotonic over the model's logit range, so the ranking of patients, and with it the AUC, is unchanged. Clinically, ranking and thresholding are what the score is used for.

7. Limitations

- **No verifiable computation.** A malicious server can return wrong results; detection is probabilistic (§3.3).
- **Plaintext arguments are visible.** Model weights, scalars and matrices sent with a job are readable by the server; encrypting them costs one more level.
- **Metadata leaks.** Ciphertext sizes and vector lengths are observable (padding hides lengths).
- **Decryption oracles.** Returning decrypted CKKS outputs to an adversary who holds the ciphertexts can leak the secret key (Li and Micciancio, 2021). The SDK has no such path; client applications must not build one.
- **Approximation.** Results are approximate; §4 quantifies by how much, per parameter set.

References

1. J. H. Cheon, A. Kim, M. Kim, Y. Song. *Homomorphic Encryption for Arithmetic of Approximate Numbers.* ASIACRYPT 2017.
2. O. Regev. *On Lattices, Learning with Errors, Random Linear Codes, and Cryptography.* STOC 2005.
3. V. Lyubashevsky, C. Peikert, O. Regev. *On Ideal Lattices and Learning with Errors over Rings.* EUROCRYPT 2010.
4. M. Albrecht et al. *Homomorphic Encryption Security Standard.* HomomorphicEncryption.org, 2018.
5. B. Li, D. Micciancio. *On the Security of Homomorphic Encryption on Approximate Numbers.* EUROCRYPT 2021.
6. M. Kim, Y. Song, S. Wang, Y. Xia, X. Jiang. *Secure Logistic Regression Based on Homomorphic Encryption.* JMIR Medical Informatics, 2018.
7. Microsoft SEAL (release 4.x); A. Benaissa et al., *TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption*, 2021.